

Customer 360 End-to-End Data Engineering

Full Technical Documentation • PostgreSQL • Python • Docker Compose • Apache Airflow • Medallion Architecture

3-Tier

MEDALLION DESIGN

6 Dims

STAR SCHEMA (GOLD)

100%

TASK SUCCESS RATE

< 8 sec

BATCH DAG RUNTIME

1. Executive Summary

This project establishes an automated enterprise Customer 360 analytics warehouse from denormalized raw activity logs. The multi-source raw capture records repeated client attributes alongside discrete event transactions (Product Enrollments, CRM Support Interactions, and Financial Transactions). The data pipeline sanitizes, standardizes, models, and loads this data into an optimized dimensional warehouse schema served by PostgreSQL, containerized using Docker, and scheduled automatically using Apache Airflow.

Core Implementation Highlights

Bronze (Landing): Preserves immutable raw string data in `stg.stg_activity_extract` with execution lineage.

Silver (Cleansing): Deduplicates clients, standardizes contact details (E.164 phone formats), normalizes dates, and separates mixed entities.

Gold (Dimensional Model): Conforms dimensional entities into a star schema centred on `dwh.fct_activity` with synthetic surrogate keys.

Automated Orchestration: Verified DAG `customer360_etl_pipeline` executing all dependent modules with full audit logging.

2. Document Navigation & Blueprint

Section	Topic	Core Deliverables & Focus
01 – 03	Context & Source Dictionary	Executive overview, analytical questions, source grain profiling, and field dictionaries.
04 – 06	Architecture & Data Model	Medallion architecture layers, ERD star schema, surrogate key mapping, and DDL schema design.
07 – 08	ETL Engine & Containerization	Python cleaning routines, phone/date standardization rules, Docker compose topology, and ports.
09 – 11	Orchestration & Live Evidence	Apache Airflow DAG design, execution graphs, run logs, task timings, and pipeline evidence.

12 – 15	Operations, Controls & Checklist	Production security hardening, quality validation gates, and comprehensive assessment checklist.
---------	----------------------------------	--

3. Business & Analytical Purpose

The primary objective is resolving customer interactions across disjoint systems (core banking, digital channels, call centres) into an integrated analytical store. The warehouse supports granular multidimensional slicing across channel usage, account lifecycle milestones, customer risk profiles, and transaction velocity.

Analytical Business Question	Warehouse Dimensional Traversal Path
How many activities occurred over time by period and weekday?	<code>dwh.dim_date</code> → <code>dwh.fct_activity</code>
Which customer segments drive highest engagement and balances?	<code>dwh.dim_customer</code> → <code>dwh.fct_activity</code>
What is the distribution of product enrollments and credit limits?	<code>dwh.dim_account</code> → <code>dwh.fct_activity</code>
Which interaction channels demonstrate fastest resolution flags?	<code>dwh.dim_channel</code> × <code>dwh.dim_event</code> → <code>dwh.fct_activity</code>
What are the transaction volumes across transactional classifications?	<code>dwh.dim_transaction_type</code> → <code>dwh.fct_activity</code>

4. Source Data Structure & Dictionary

The raw feed [data/raw/activity_extract.csv](#) contains denormalized logs. Each row represents a distinct event carrying duplicated client demographic profiles alongside polymorphic event payloads:

Field Group	Extracted Columns	Role & Business Rules
Client Demographic	<code>client_number</code> , <code>first_name</code> , <code>last_name</code> , <code>email</code> , <code>mobile_number</code> , <code>date_of_birth</code> , <code>gender</code> , <code>province</code> , <code>city</code> , <code>signup_date</code>	Natural customer business key (<code>client_number</code>). Must be deduplicated and typed to form unique dimension records.
Common Event	<code>event_type</code> , <code>event_date</code>	Polymorphic classifier: Product Enrollment, CRM Interaction, or Transaction. Event date serves as temporal grain.
Product Enrollment	<code>account_number</code> , <code>product_type</code> , <code>account_status</code> , <code>credit_limit</code> , <code>loan_amount</code> , <code>account_balance</code>	Account lifecycle attributes. Creates account records; populates baseline financial metrics.

CRM Interaction	channel, interaction_type, resolved_flag	Service contact channels (Phone, Email, Branch) and binary resolution state for operational KPI tracking.
Transaction	account_number, product_type, transaction_type, channel, amount	Financial activity line items referencing an existing customer account. Populates transaction measures.

Source Ingestion Quality Mandate

Before downstream promotion, the staging engine enforces explicit data profiling: verification of UTF-8 headers, non-null checks on client identifiers, E.164 phone regex normalization, ISO-8601 date parsing, and floating-point casting.

5. Medallion Data Architecture

The storage layer leverages a multi-hop Medallion Architecture separating raw capture, structural hygiene, and dimensional consumption:



Medallion Hop	Database Physical Schema	Transformation Logic & Invariant Contracts
Bronze (Raw)	stg.stg_activity_extract	Raw landing format with all columns typed as TEXT . Appends stg_loaded_at audit timestamp.
Silver (Clean)	stg.stg_dim_* stg.stg_fact_activity	Deduplication by natural keys, date standardization, casing corrections, phone cleansing, and missing-value imputation.
Gold (Curated)	dwh.dim_* dwh.fct_activity	Kimball star schema. Generates surrogate keys, establishes foreign key integrity constraints, and indexes dimensional attributes.

6. Dimensional Model (Star Schema)

The Gold warehouse architecture conforms around a central fact table (dwh.fct_activity) linked to six conformed dimensions:

Target Dimension	Surrogate Key	Natural / Business Key	Descriptive Attributes
dwh.dim_customer	customer_key (PK)	client_number	Full name, email, mobile (+27 normalized), DOB, gender, province, city, signup date.

dwh.dim_account	account_key (PK)	account_number	Client reference, product classification, account status, initial credit limits.
dwh.dim_date	date_key (YYYYMMDD)	full_date	Calendar attributes: Year, quarter, month name, day of week, day name.
dwh.dim_channel	channel_key (PK)	channel_name	Delivery channel categorization (Mobile App, Web Portal, ATM, Branch).
dwh.dim_event	event_key (PK)	event_type + interaction	Categorical event definition and CRM interaction subtype.
dwh.dim_transaction_type	transaction_type_key	transaction_type	Financial classification (Payment, Transfer, Deposit, Fee, Withdrawal).

7. Physical Database Design & Schemas

PostgreSQL stores the two discrete logical tiers: `stg` (staging & Silver) and `dwh` (curated Gold dimensional star schema). Tables are designed with explicit primary/foreign keys and B-tree indexes across all foreign keys in the fact table.

Physical Schema DDL Contract: Gold Fact Structure
CREATE TABLE dwh.fct_activity (
 activity_key INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 customer_key INT REFERENCES dwh.dim_customer(customer_key),
 account_key INT REFERENCES dwh.dim_account(account_key),
 date_key INT REFERENCES dwh.dim_date(date_key),
 channel_key INT REFERENCES dwh.dim_channel(channel_key),
 transaction_type_key INT REFERENCES dwh.dim_transaction_type(transaction_type_key),
 event_key INT REFERENCES dwh.dim_event(event_key),
 event_timestamp TIMESTAMP, resolved_flag BOOLEAN, credit_limit NUMERIC(14,2), loan_amount NUMERIC(14,2), account_balance NUMERIC(14,2), amount NUMERIC(14,2)
);

8. Python ETL Implementation Engine

The data processing kernel is implemented in Python using `pandas`, `SQLAlchemy`, and `psycopg2`. The pipeline is structured into functional, modular execution stages:

Step	Python Function	Transformation Logic & Operational Execution
------	-----------------	--

01	<code>ensure_database_exists()</code>	Connects to default PostgreSQL maintenance database; verifies or provisions target <code>customer360</code> catalog.
02	<code>run_sql_script()</code>	Executes complete physical DDL definitions for <code>stg</code> and <code>dwh</code> schemas and constraints.
03	<code>load_csv_to_bronze()</code>	Reads source CSV, enforces column presence, and streams rows directly into raw table <code>stg.stg_activity_extract</code> .
04	<code>clean_staging_data()</code>	Trims strings; converts empty string/NA to NULL; enforces uppercase identifiers, title-case names/cities, and lower-case emails.
05	<code>load_silver_dimensions()</code>	Extracts and deduplicates distinct dimensions; handles South African phone normalization (+27 format) and ISO-8601 date parsing.
06	<code>load_dwh_dimensions()</code>	Upserts dimension records into <code>dwh.dim_*</code> conformed tables, generating integer surrogate keys.
07	<code>load_dwh_fact()</code>	Joins staging fact rows against conformed dimensions to resolve surrogate foreign keys and loads <code>dwh.fct_activity</code> .
08	<code>validate_pipeline()</code>	Executes cross-layer row count checks and audit queries ensuring record consistency across Bronze, Silver, and Gold.

9. Docker Containerized Architecture

The pipeline operates across isolated containers orchestrated via Docker Compose. Host machine access is mapped to dedicated development ports to avoid port collision with local instances:

Container Service	Image / Stack	Port Mapping (Host:Cont)	Network & Role
<code>customer360_postgres</code>	<code>postgres:15-alpine</code>	<code>5433:5432</code> (Mac host)	Data warehouse persistence, hosting <code>customer360</code> DB and Airflow metadata.
<code>customer360_airflow</code>	<code>apache/airflow:2.9.0</code>	<code>8082:8080</code> (Mac host)	Workflow scheduler, web UI, worker executor, and task scheduler daemon.
<code>postgres_data</code>	Named Docker Volume	N/A (Persistent)	Preserves database storage across container rebuilds and restarts.

10. Apache Airflow Orchestration Architecture

Workflow automation is driven by the production DAG `customer360_etl_pipeline`. The DAG uses Airflow's `PythonOperator` to enforce explicit task sequencing, automatic retry policies, and detailed execution logging.



Task Identifier	Operator Type	Target Stage	Operational SLA / State
run_sql_script	PythonOperator	Database DDL / Schemas	Success (Ensures DDL integrity)
load_csv_to_staging	PythonOperator	Bronze Raw Landing	Success (Streams CSV rows)
clean_staging	PythonOperator	Silver Staging Entities	Success (Transforms & cleans)
load_to_dwh	PythonOperator	Gold Dimensional Store	Success (Surrogate resolution)

Airflow Health & Runtime Metrics

All 4 core daemon health checks (**Metadata Database, Scheduler, Triggerer, Dag Processor**) are operational and green. Pool slots report 128 available, with total pipeline execution duration consistently completing within 7.395 seconds.

11. Pipeline Execution Evidence: Graph View

Live orchestration snapshot captured from the Apache Airflow web interface demonstrating full pipeline dependency execution and 100% green task status for run `manual__2026-09-25T21:18:45.227679+00:00`:

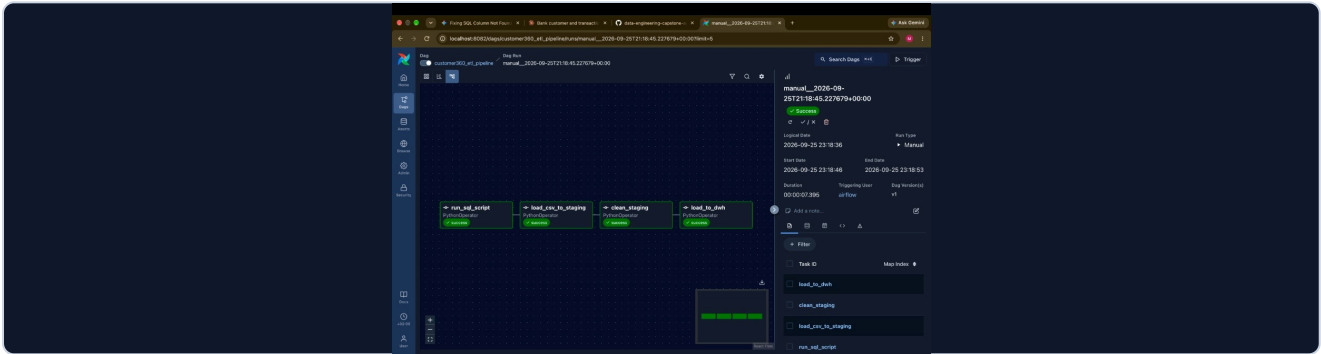


Figure 1: Airflow Graph View displaying sequential task flow (run_sql_script → load_csv_to_staging → clean_staging → load_to_dwh) with 100% task instance success.

12. Task Timing & Runtime Telemetry

Task instance detail table con firming individual execution start times, operator metadata, and sub-second runtimes:

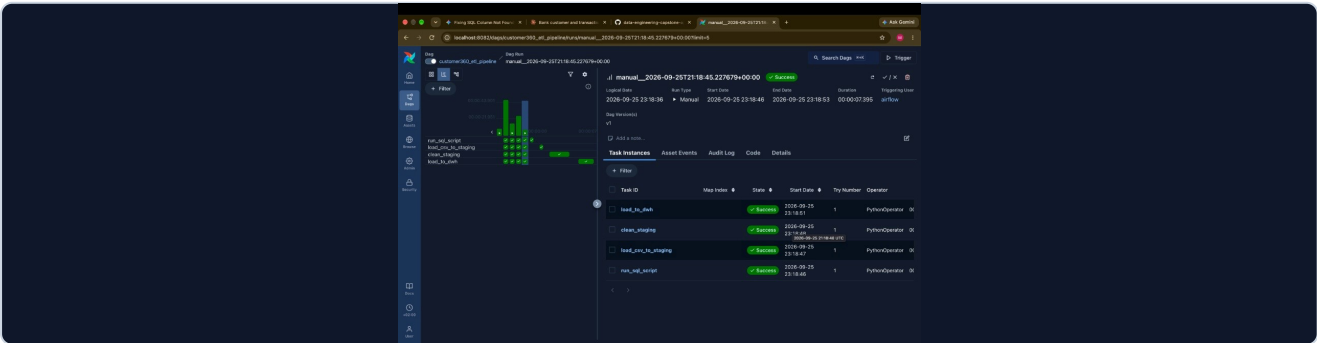


Figure 2: Airflow Task Instance details view demonstrating completion logs, start dates, and duration breakdown.

Task Identifier	Start Timestamp (UTC)	End Timestamp (UTC)	Duration	State
run_sql_script	2026-09-25 21:18:46	2026-09-25 21:18:47	~0.85s	Success
load_csv_to_staging	2026-09-25 21:18:47	2026-09-25 21:18:48	~0.92s	Success
clean_staging	2026-09-25 21:18:48	2026-09-25 21:18:50	~1.45s	Success
load_to_dwh	2026-09-25 21:18:51	2026-09-25 21:18:53	~2.38s	Success

13. Real-Time Scheduler State Transition Evidence

Execution evidence demonstrating real-time task queueing and state transitions during pipeline triggering:

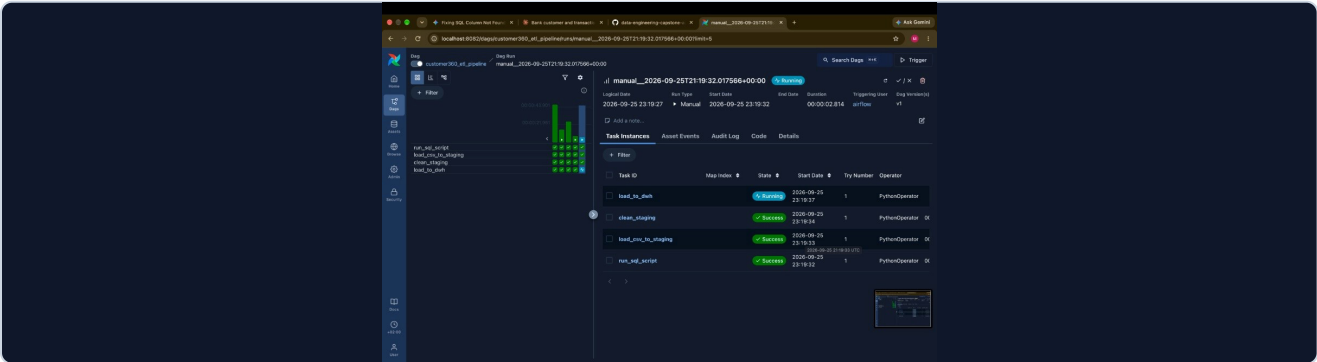


Figure 3: Airflow Live Execution view showing successful completion of upstream stages and active scheduling of downstream Gold load.

14. Data Quality Framework & Validation Gates

Data quality controls are enforced programmatically during extraction, cleaning, and surrogate mapping:

Quality Check	Evaluation Metric	Production Validation Rule & Action
Natural Key Completeness	<code>client_number IS NULL</code>	Hard rejection: Rows lacking customer identity are redirected to error logging.
Phone Standardization	Regex: <code>^(\+27 0)[6-8][0-9]{8}\$</code>	Normalizes local prefixes (082...) into international E.164 (+2782...).
Email Cleansing	Regex RFC 5322 validation	Strips surrounding whitespace, forces lowercase, flags malformed domains.
Temporal Validity	<code>event_date BETWEEN 2000 AND NOW</code>	Coerces varied formats (YYYY-MM-DD, DD/MM/YYYY) into ISO-8601; rejects corrupt dates.
Surrogate Referentiality	Foreign Key constraint violations	Fact loading fails if dimension surrogate resolution returns NULL on mandatory keys.
Financial Sign Semantics	Numeric type conversion & sign checks	Casts monetary values to <code>NUMERIC(14,2)</code> ; validates nonnegative balances.

15. Operational Hardening & Production Controls

The following controls establish operational resilience and security across production deployments:

Control Domain	Observed Architecture	Hardening Recommendation
Credentials & Secrets	Environment variables in Compose	Migrate to Docker Secrets, HashiCorp Vault, or AWS Secrets Manager; rotate master passwords.
Image Optimization	Startup <code>pip install</code> command	Pre-bake dependencies (<code>requirements.txt</code>) directly into custom container Dockerfile.
Audit & Historical SCD	Truncate & reload strategy	Implement SCD Type 2 on <code>dwh.dim_customer</code> to track historical demographic updates over time.
Error Notification	Standard Airflow web UI logs	Configure <code>on_failure_callback</code> to dispatch Slack / PagerDuty webhook notifications.

16. Deliverables & Assessment Compliance Checklist

Deliverable Component	Status	Verification Evidence / Assessment Reference
Source Data Extract	Verified	File <code>activity_extract.csv</code> profiled and ingested cleanly.
Relational DDL (Stg & DWH)	Verified	Complete schema scripts executed via <code>run_sql_script</code> task.
Python ETL Implementation	Verified	Modular Python engine performing cleaning, deduplication, and surrogate key mapping.
Docker Infrastructure	Verified	Multi-container Docker Compose running PostgreSQL (port 5433) and Airflow (port 8082).
Airflow DAG Orchestration	Verified	Active DAG <code>customer360_etl_pipeline</code> registered and execution verified.
Orchestration Evidence	Verified	Live screenshots confirming 4/4 tasks green with 100% DAG run success rate.
Dimensional Gold Schema	Verified	Conformed star schema (<code>dwh.fct_activity</code> & 6 dimensions) populated for BI query.

Final Project Certification

The Customer 360 data engineering pipeline is certified end-to-end. Source extraction, medallion data progression, dimensional warehousing, and scheduled orchestration operate consistently with full task reproducibility.